# Heurísticas para o Problema de Orientação Agrupada Generalizada

**Gabriel Coelho dos Santos**[1], **Inaue Ferreira da Silva**[1], **Kayque Meira Siqueira**[1], **Lucas Fernandes Santos**[1], **Rubens da Cunha Castro**[1]

[1] Instituto de Ciências Exatas
Universidade Federal de Minas Gerais

## 1. Introdução

O planejamento de rotas surge sempre que é preciso visitar diferentes locais sob limitações de tempo, distância percorrida ou outros recursos. Em muitos cenários não é possível visitar todos os pontos, de modo que é preciso decidir quais locais visitar e em que ordem, buscando maximizar o benefício obtido com os recursos disponíveis. O Problema de Orientação Agrupada Generalizada (GCOP, do inglês *Generalized Clustered Orienteering Problem*), definido por Álvarez-Miranda et al. [1], busca resolver um desses cenários. Os autores se inspiraram no Pokémon GO, um jogo de realidade aumentada baseado em geolocalização, em que criaturas virtuais aparecem em locais do mundo real, e o jogador precisa se deslocar até elas para capturá-las. O problema então é planejar um passeio que capture as espécies desejadas e que caiba no tempo e bateria disponíveis do usuário.

O GCOP é definido sobre um grafo completo não direcionado $G = (V, E)$, com $n = |V|$ vértices. Temos como entradas uma distância $d_e \geq 0$ para cada aresta $e \in E$ (não necessariamente respeita a desigualdade triangular); um prêmio $p_i \geq 0$ para cada vértice $i \in V$; uma coleção $\mathcal{C} = \{C_1, \ldots, C_K\}$ de *clusters* $C_k \subseteq V$, que não precisam ser disjuntos nem cobrir $V$ (vértices fora de qualquer *cluster* são opcionais); e um orçamento $B$ para o comprimento da rota. O objetivo é encontrar um ciclo que maximiza a soma dos prêmios dos vértices visitados, respeitando as seguintes restrições: (i) ao menos um vértice de cada *cluster* é visitado; (ii) o comprimento total da rota não excede $B$. Para isso, devemos decidir se cada vértice é visitado ou não, e se cada aresta faz da rota. A rota não tem ponto de partida fixo e, como cada aresta é usada no máximo uma vez, tem pelo menos três vértices.

O GCOP combina dois problemas clássicos: o Problema de Orientação (OP), em que se busca uma rota que maximize o prêmio coletado sem exceder um limite de comprimento, e o Problema do Caixeiro-Viajante Generalizado (GTSP), em que a rota deve visitar exatamente um vértice de cada *cluster*. Quanto à sua classe de complexidade, note que o OP é um caso particular do GCOP, obtido com um *cluster* formado apenas pelo vértice raiz. Como o OP é NP-difícil, o GCOP também é NP-difícil.

Estudar o GCOP vale a pena porque o modelo descreve diretamente uma atividade de milhões de pessoas e se estende a problemas logísticos relevantes. O Pokémon GO ultrapassou 100 milhões de instalações em seu primeiro mês e deu origem a negócios paralelos, como passeios de táxi por locais de captura [1]. No modelo, cada espécie desejada é um *cluster* formado pelos locais onde ela aparece, o orçamento representa o tempo, a bateria ou a franquia de dados, e a ausência de ponto de partida reflete o fato de que jogadores frequentemente caminham em grupo. Fora do jogo, o mesmo modelo

representa a distribuição para redes varejistas com vários pontos de venda, rotas que garantam presença em todas as classes de clientes e o planejamento de serviços móveis de saúde [1].

Este trabalho pretende propor, implementar e avaliar heurísticas para o GCOP. Especificamente, pretende-se: (i) propor uma busca local que calcula exatamente a variação de comprimento ao inserir, remover e trocar vértices, em substituição ao índice aproximado usado pela heurística do artigo base; (ii) implementar cinco métodos baseados nessa busca local, a saber, MSPD-LS, *Variable Neighborhood Search* (VNS), *Iterated Local Search* (ILS), *Adaptive Large Neighborhood Search* (ALNS) e Algoritmo Memético (MA), além de uma variante do ILS (ILS-C) que investiga o efeito de trocar o representante de um *cluster*; e (iii) compará-los com a heurística *multi-start* com distâncias perturbadas (MSPD) do artigo base, usada como controle, em instâncias derivadas da TSPLIB e em instâncias sintéticas no estilo Pokémon GO, usando uma formulação exata como referência nas instâncias menores.

Atingir esses objetivos é importante porque hoje não há método capaz de obter rapidamente boas soluções para as instâncias grandes do GCOP. Os métodos de Álvarez-Miranda et al. [1] são exatos e, mesmo com uma hora de execução, não provam a otimalidade de várias das maiores instâncias TSPLIB (com mais de 400 vértices) nem de 11 das 18 instâncias Pokémon com orçamento elevado. A única heurística existente é o procedimento *multi-start* que inicializa o método exato, e os próprios autores apontam o desenvolvimento de (meta)heurísticas como trabalho futuro. Heurísticas que obtenham boas soluções em poucos segundos tornam o modelo utilizável em instâncias maiores e em situações que exigem resposta rápida, como planejar um passeio no momento em que o jogador decide sair.

O restante do texto está organizado da seguinte forma. A Seção 2 situa o trabalho no estado da arte. A Seção 3 descreve as heurísticas propostas e de controle, o ambiente experimental e as hipóteses investigadas. A Seção 4 apresenta as etapas, as atribuições da equipe e os prazos.

## 2. Trabalhos relacionados

O artigo de Álvarez-Miranda et al. [1] é a base deste projeto, pois definiu o GCOP e fornece o controle e as instâncias TSPLIB usados aqui. Os autores propuseram formulações inteiras e algoritmos *branch-and-cut*, inicializados por uma heurística *multi-start*, e provaram a otimalidade da maior parte das instâncias em uma hora, mas não das maiores. Concluíram que (meta)heurísticas são necessárias para instâncias maiores, lacuna que este projeto aborda.

O trabalho de Pěnička et al. [9] trata do *Set Orienteering Problem* [2], o problema mais próximo do GCOP, em que visitar um vértice de um *cluster* rende o prêmio do *cluster*, em vez de ser uma exigência. O VNS proposto pelos autores obteve resultados competitivos com a literatura nesse problema e em outras variantes do OP. A conclusão de que a troca sistemática de vizinhanças é eficaz para escolher o representante de cada *cluster* fundamenta o VNS proposto.

O trabalho de Vansteenwegen et al. [14] fornece a perturbação do ILS proposto. Aplicado ao *Team Orienteering Problem with Time Windows*, seu ILS, que remove visitas

consecutivas e reinsere vértices de forma gulosa, obteve soluções próximas das melhores conhecidas em poucos segundos. Os autores concluíram que uma perturbação simples basta para problemas de orientação, o que é compatível com o objetivo de resposta rápida deste projeto.

O trabalho de Røpke e Pisinger [10] introduz o ALNS, cujo esquema é seguido pelo ALNS proposto. Aplicado à coleta e entrega com janelas de tempo, o método escolhe operadores de destruição e reconstrução pelo desempenho histórico e melhorou muitas das melhores soluções conhecidas. Os autores concluíram que a seleção adaptativa torna o método robusto a instâncias diferentes, propriedade útil aqui, já que as instâncias TSPLIB e Pokémon diferem bastante.

Este projeto se diferencia dessa literatura por propor as primeiras metaheurísticas específicas para o GCOP. Elas podem superar a heurística do artigo base porque calculam exatamente o custo de inserções, remoções e trocas, em vez de estimá-lo, e porque usam operadores que trocam o representante de um *cluster*.

## 3. Metodologia

### 3.1. Descrição das Heurísticas Propostas

Os cinco métodos propostos (MSPD-LS, VNS, ILS, ALNS e MA) compartilham a notação, a busca local e os procedimentos auxiliares descritos a seguir, de modo que as diferenças entre eles estejam apenas na estratégia de busca. Uma solução $T$ é uma sequência cíclica de vértices distintos, com comprimento $L(T)$ e prêmio $P(T)$; ela é *viável* se cobre todos os *clusters* e $L(T) \leq B$, e é *melhor* que outra se tem maior prêmio ou, em caso de empate, menor comprimento. Um vértice $v \in T$ é *removível* se todo *cluster* que o contém tem outro vértice em $T$. Para $v \notin T$, $\Delta^+(v) = \min_{(a,b)}(d_{av} + d_{vb} - d_{ab})$ é o aumento exato do comprimento ao inserir $v$ na melhor posição entre vértices consecutivos $a$ e $b$; para $v \in T$, com vizinhos $a$ e $b$, $\Delta^-(v) = d_{av} + d_{vb} - d_{ab}$ é a redução exata ao retirá-lo. Sem a desigualdade triangular, $\Delta^+$ pode ser negativo e $\Delta^-$ também (retirar o vértice alonga a rota); por isso, as razões usadas são $\rho^+(v) = p_v / \max\{\Delta^+(v), \varepsilon\}$ e $\rho^-(v) = \Delta^-(v) / \max\{p_v, \varepsilon\}$, com $\varepsilon = 10^{-6}$.

A busca local proposta (BL, Algoritmo 1) é o componente que todos os métodos compartilham e alterna três movimentos, por no máximo 50 rodadas, até que nenhum melhore a solução. OTIMIZAROTA aplica 2-opt [3] e Or-opt [8], com primeira melhora, para encurtar a rota sem alterar os vértices visitados, liberando orçamento. INSERERAZAO, enquanto algum vértice de fora couber em $B$, insere na melhor posição primeiro os vértices com $\Delta^+ < 0$ e depois o de maior $\rho^+$ [13]. TROCA, com melhor melhora, substitui $u \in T$ por $v \notin T$ na melhor posição se o resultado couber em $B$, mantiver todos os *clusters* cobertos e for melhor; como o critério desempata pelo comprimento, ela também aceita um vértice de mesmo prêmio que encurte a rota, essencial nas instâncias de prêmio uniforme. A diferença em relação ao refinamento do artigo base (Seção 3.2) é que todos os custos são exatos, e um *cache* das melhores posições de inserção, atualizado só nas arestas alteradas, evita recalculá-los a cada movimento.

Quatro procedimentos auxiliares constroem, perturbam e reparam soluções. REPARA$(T, X)$ torna a solução viável sem usar os vértices do conjunto proibido $X$: cobre cada *cluster* descoberto com o vértice de menor $\Delta^+$ fora de $X$; enquanto $L(T) > B$,

remove o vértice removível de maior $\rho^-$ e aplica OTIMIZAROTA; e, se não houver mais vértice removível, declara a solução inviável, que é descartada. Como só vértices removíveis saem, o procedimento sempre termina. INICIAL é a melhor entre as soluções da inserção NI, da inserção FI [4] e da árvore de Steiner (ST) [12], cada uma seguida de REPARA e BL. GRASP constrói como a NI, mas sorteia cada próximo *cluster* entre os três mais próximos da rota e cada inserção entre os três vértices de maior $\rho^+$. AGITA$(T, k)$ remove $k$ vértices sorteados, insere $k$ vértices de fora sorteados e aplica REPARA proibindo os vértices removidos, o que força a troca do representante de um *cluster* descoberto.

O VNS (Algoritmo 2) [6] explora vizinhanças de intensidade crescente $N_1, \ldots, N_6$, em que $N_k$ é AGITA$(T, k)$, com vértices removidos em qualquer posição da rota [11, 9]. Se a perturbação seguida de BL produz uma solução melhor, ela é aceita e a busca volta a $N_1$; caso contrário, passa à vizinhança seguinte, e após $N_6$ o ciclo recomeça.

O ILS (Algoritmo 3) [5] perturba o ótimo local atual e aplica a busca local novamente; seguindo Vansteenwegen et al. [14], a perturbação remove $m$ vértices **consecutivos** a partir de uma posição sorteada e aplica REPARA proibindo-os. O tamanho $m$ cresce a cada falha, volta a 1 quando ultrapassa $|T|/3$ ou quando a melhor solução melhora, toda solução viável substitui a corrente (passeio aleatório) e, a cada 100 falhas seguidas, a busca retorna à melhor solução. O **ILS-C**, usado apenas na hipótese H4 e não contado entre os métodos propostos, é idêntico ao ILS, exceto que, com probabilidade 0,5, a perturbação remove os vértices de $\lceil m/2 \rceil$ *clusters* sorteados em vez de vértices consecutivos, forçando a troca de representantes em qualquer região da rota.

O ALNS (Algoritmo 4) [10] destrói, a cada iteração, uma fração $q \in [0{,}10;\ 0{,}40]$ dos vértices da rota e a reconstrói com operadores escolhidos de forma adaptativa. Os cinco operadores de destruição são *aleatório*, *pior razão* (maior $\rho^-$), *segmento* (vértices consecutivos), *radial* (os mais próximos de um vértice sorteado) e *cluster*, específico do GCOP, que remove os vértices de *clusters* sorteados; os vértices removidos formam o conjunto proibido $X$ e não podem ser reinseridos na mesma iteração, nem pela reconstrução nem por REPARA. Os três de reconstrução são *guloso* (cobertura pelo menor $\Delta^+$ seguida de INSERERAZAO), *regret* (cobre primeiro o *cluster* com maior diferença entre suas duas inserções mais baratas) e *ruidoso* (guloso com $\rho^+$ multiplicada por um fator em $[0{,}8;\ 1{,}2]$). Os operadores são sorteados por roleta e pontuados com 33, 9 ou 13 pontos quando geram, respectivamente, uma nova melhor solução, uma solução melhor que a atual ou uma solução pior aceita; a cada 100 iterações, peso $\leftarrow 0{,}9\,\text{peso} + 0{,}1\,\pi/\theta$, em que $\pi$ é a pontuação acumulada e $\theta$ o número de usos. Soluções de menor prêmio são aceitas com probabilidade $e^{(P(T')-P(T))/\Theta}$, com $\Theta = \max\{\Theta_0\,(1 - t/t(n)),\ 10^{-6}\}$ e $\Theta_0 = 0{,}05\,P(\text{INICIAL})/\ln 2$, de modo que, no início, uma solução com prêmio 5% menor é aceita com probabilidade 0,5; soluções de mesmo prêmio só são aceitas se não forem mais longas.

O MA (Algoritmo 5) [7] mantém uma população de 10 soluções (INICIAL e 9 obtidas por GRASP, REPARA e BL) e gera filhos por cruzamento, aplicando a busca local a cada um. Os pais $P_1$ e $P_2$ são escolhidos por torneio binário; no cruzamento por *tour gigante*, o filho recebe um trecho contíguo de 30% a 70% da rota de $P_1$ seguido dos vértices de $P_2$ que ainda não tem, na ordem de $P_2$, e OTIMIZAROTA e REPARA o tornam viável. Com probabilidade 0,2, aplica-se a mutação AGITA$(F, 2)$. O filho substitui o pior

indivíduo se for melhor que ele e nenhum indivíduo visitar exatamente o mesmo conjunto de vértices.

O MSPD-LS é a ablação que isola o efeito da busca local: é idêntico ao controle MSPD (Algoritmo 6), com a mesma construção, a mesma perturbação das distâncias e o mesmo tempo, exceto que a fase de melhoria (linha 8) usa REPARA seguido de BL no lugar de OTIMIZAROTA seguido de REFINOSIGMA. Assim, qualquer diferença de desempenho entre os dois pode ser atribuída à BL como um todo, isto é, ao cálculo exato dos custos e aos seus movimentos em conjunto.

### 3.2. Descrição das Heurísticas de Controle

A heurística de controle é o *multi-start* com distâncias perturbadas (MSPD, Algoritmo 6), procedimento que inicializa o método exato de [1] e única heurística da literatura para o GCOP. A cada iteração, as distâncias são multiplicadas por fatores aleatórios entre 1 e 1,2, para que a construção gulosa não produza sempre a mesma rota, e constrói-se uma rota a partir de um vértice sorteado, sorteando entre a NI [4] e a ST [12]. A rota é reavaliada com as distâncias reais e melhorada pelo refinamento $\sigma$ (Algoritmo 7), que remove, insere e troca vértices com base no índice $\sigma_i = \min_{j \in V_T \setminus \{i\}} d_{ij}/p_i$, a distância ao vértice visitado mais próximo dividida pelo prêmio, que estima o custo de um vértice pela proximidade da rota, e não pela variação real do comprimento. O artigo base executa 10 iterações; aqui, para uma comparação justa, o MSPD será executado pelo mesmo tempo $t(n)$ dos métodos propostos.

### 3.3. Ambiente de Execução dos Experimentos

#### Descrição do ambiente de experimentação

O ambiente de experimentação, o protocolo de execução e as medidas comuns a todas as hipóteses são os seguintes. Os algoritmos serão implementados em C++17 (`g++ 14.2.0`, opções `-O2 -std=c++17`), a formulação exata usada como referência na hipótese H3 será resolvida pelo *solver* HiGHS 1.15.1 com uma *thread*, e a análise será feita em Python 3.11, em um AMD Ryzen 5 5500 (6 núcleos, 3,6 GHz) com 16 GB de RAM e Windows 10 Pro (64 bits). Cada execução usará uma *thread*, com até 5 execuções simultâneas, deixando um núcleo livre para o sistema. Todas as heurísticas terão o mesmo limite de tempo de relógio $t(n) = \max(1, n/50)$ s, incluindo a construção inicial, e serão executadas com as mesmas 10 sementes por instância; seus parâmetros são os valores fixados na Seção 3.1, tomados da literatura e não ajustados nas instâncias de teste. A qualidade é medida pelo *gap* ao BKS (*best known solution*), $100\,(BKS - P)/BKS$, em que BKS é o maior prêmio obtido por qualquer execução ou pela formulação exata e $P$ é o prêmio da execução (100% se ela não encontrar solução viável); o *gap* ao ótimo (OPT) é definido da mesma forma quando a formulação exata prova a otimalidade. A unidade de análise é o *gap* médio das 10 sementes em cada instância, e as comparações usam o teste de Wilcoxon pareado por instância (empates pelo método de Pratt) e o de Friedman, com significância de 5% e correção de Holm em cada família de comparações.

**Algoritmo 1. Busca local proposta (BL)**

**procedure** BL($T$)
  **repeat** (no máximo 50 vezes)
    OTIMIZAROTA($T$)
    $a \leftarrow$ INSEREAZAO($T$)
    $b \leftarrow$ TROCA($T$)
  **until** não $a$ e não $b$
**end procedure**

**Algoritmo 2. VNS**

**procedure** VNS(tempo)
  $T \leftarrow$ INICIAL; $k \leftarrow 1$
  **while** houver tempo **do**
    $T' \leftarrow$ AGITA($T, k$); BL($T'$)
    **if** $T'$ é viável e melhor que $T$ **then**
      $T \leftarrow T'$; $k \leftarrow 1$
    **else**
      $k \leftarrow (k \bmod 6) + 1$
    **end if**
  **end while**
  **return** $T$
**end procedure**

**Algoritmo 3. ILS e ILS-C**

**procedure** ILS(tempo)
  $T \leftarrow$ INICIAL; $T^* \leftarrow T$; $m \leftarrow 1$; $f \leftarrow 0$
  **while** houver tempo **do**
    $X \leftarrow m$ vértices consecutivos de $T$ (ILS-C: com prob. 0,5, os de $\lceil m/2 \rceil$ *clusters*)
    $T' \leftarrow T \setminus X$; REPARA($T', X$); BL($T'$)
    **if** $T'$ é viável e melhor que $T^*$ **then**
      $T^* \leftarrow T \leftarrow T'$; $m \leftarrow 1$; $f \leftarrow 0$
    **else**
      **se** $T'$ é viável: $T \leftarrow T'$
      $m \leftarrow m + 1$ (volta a 1 se $m > |T|/3$)
      $f \leftarrow f + 1$; **se** $f \bmod 100 = 0$: $T \leftarrow T^*$
    **end if**
  **end while**
  **return** $T^*$
**end procedure**

**Algoritmo 4. ALNS**

**procedure** ALNS(tempo)
  $T \leftarrow$ INICIAL; $T^* \leftarrow T$; pesos $\leftarrow 1$
  **while** houver tempo **do**
    sorteia $d$, $r$ pela roleta e $q \in$ [0,10; 0,40]
    $(T', X) \leftarrow d(T, q)$ ▷ $X$: removidos
    $T' \leftarrow r(T', X)$; REPARA($T', X$); OTIMIZAROTA($T'$)
    $\Theta \leftarrow \max\{\Theta_0\,(1 - t/t(n)),\ 10^{-6}\}$; $U \leftarrow$ sorteio em $[0, 1]$
    **if** $T'$ é inviável **then**
      descarta $T'$
    **else if** $T'$ é melhor que $T^*$ **then**
      BL($T'$); $T^* \leftarrow T \leftarrow T'$; pontua 33
    **else if** $T'$ é melhor que $T$ **then**
      $T \leftarrow T'$; pontua 9
    **else if** $P(T') < P(T)$ e $U < e^{(P(T')-P(T))/\Theta}$ **then**
      $T \leftarrow T'$; pontua 13
    **end if**
    a cada 100 iterações: atualiza pesos
  **end while**
  **return** $T^*$
**end procedure**

**Algoritmo 5. Algoritmo memético (MA)**

**procedure** MA(tempo)
  $Pop \leftarrow$ INICIAL + 9 GRASP com REPARA e BL
  **while** houver tempo **do**
    $P_1 \leftarrow$ TORNEIO($Pop$); $P_2 \leftarrow$ TORNEIO($Pop$)
    $F \leftarrow$ trecho de $P_1$ + resto de $P_2$
    OTIMIZAROTA($F$); REPARA($F$)
    com prob. 0,2: $F \leftarrow$ AGITA($F, 2$)
    BL($F$)
    **if** $F$ é viável, supera o pior e é inédito **then**
      substitui o pior por $F$
    **end if**
  **end while**
  **return** a melhor solução de $Pop$
**end procedure**

**Algoritmo 6. MSPD (controle) [1]**

```
procedure MSPD(tempo)
    T* ← ∅
    while houver tempo do
        d̃_ij ← d_ij · U[1; 1,2], ∀{i, j}
        s ← vértice aleatório
        T ← NI ou ST com d̃, a partir de s
        reavalia T com as distâncias reais d
        OTIMIZAROTA(T); REFINOSIGMA(T)
        if T é viável e melhor que T* then
            T* ← T
        end if
    end while
    return T*
end procedure
```

**Algoritmo 7. Refinamento $\sigma$ (controle) [1]**

```
procedure REFINOSIGMA(T)
    for rodada ← 1 até 10 do
        enquanto L(T) > B: remove maior σ
        cobre clusters pelo menor σ
        insere por σ crescente, se couber
        troca por vértices de σ menor
        OTIMIZAROTA(T)
        if não houve melhoria then
            pare
        end if
    end for
end procedure
```

### Descrição das instâncias de teste

Serão usados dois conjuntos de instâncias, com características opostas, totalizando 408 instâncias. O conjunto **TSPLIB-GCOP** (*clusters* compactos, distâncias métricas) usa os 65 grafos do artigo base [1], derivados da TSPLIB ($14 \leq n \leq 493$), com distâncias euclidianas e $\lceil n/5 \rceil$ *clusters* disjuntos [4]; o orçamento é $B = \alpha \cdot TSP + (1-\alpha) \cdot GTSP$, com $\alpha \in \{0{,}25;\ 0{,}5;\ 0{,}75\}$, em que $TSP$ e $GTSP$ são os comprimentos ótimos ou melhores conhecidos das rotas por todos os vértices e por um vértice de cada *cluster*, o que garante $B \geq GTSP$ e, portanto, viabilidade; com prêmios uniformes ($p_i = 1$) ou pseudoaleatórios ($p_i = 1 + (7213\, i \bmod 100)$), são 390 instâncias. O conjunto **POKE** (*clusters* espalhados, distâncias não métricas) substitui as instâncias Pokémon do artigo base, que não foram disponibilizadas, e por isso seus resultados não são comparáveis aos do artigo: serão geradas 3 cidades com 450 locais em 4 km $\times$ 4 km, concentrados em pontos de interesse, cada local com uma de 100 espécies, das quais 15 são obrigatórias e formam os $K = 15$ *clusters*; a distância é a euclidiana multiplicada por um fator de desvio de rua sorteado em $[1{,}15;\ 1{,}40]$ para cada par, o que viola a desigualdade triangular, e cidades em que a construção NI seguida de OTIMIZAROTA não encontre uma rota de até 10 km cobrindo os *clusters* são refeitas, o que garante a viabilidade de todos os orçamentos. Com $B \in \{10, 15, 20\}$ km e prêmios uniformes ou inversamente proporcionais à frequência da espécie, são 18 instâncias. Para a análise, as instâncias formam cinco classes: T1 ($n \leq 52$), T2 ($53 \leq n \leq 101$), T3 ($102 \leq n \leq 200$) e T4 ($201 \leq n \leq 493$), da TSPLIB-GCOP, e PK (POKE).

### 3.3.1. Hipótese 1: Busca local proposta versus refinamento $\sigma$ no MSPD

A hipótese H1 afirma que, com o mesmo *multi-start* e o mesmo tempo, trocar o refinamento $\sigma$ pela BL reduz o *gap* ao BKS, porque o índice $\sigma$ aproxima mal o custo real de inserção em rotas longas ou com distâncias não métricas.

Para testá-la, MSPD e MSPD-LS serão executados nas 408 instâncias, coletando-se o *gap* ao BKS, o número de iterações do laço principal e o tempo médio de uma fase de melhoria (ms); as duas últimas medidas indicam se um eventual ganho vem de uma

melhoria mais eficaz ou de mais reinícios.

Os resultados serão apresentados em uma tabela com uma linha por classe (T1 a T4, PK e total) e colunas com o *gap* médio, as iterações médias e o tempo da fase de melhoria de cada método, as vitórias/empates/derrotas do MSPD-LS e o valor-$p$, e em um gráfico de dispersão com o *gap* do MSPD no eixo $x$, o do MSPD-LS no eixo $y$, um ponto por instância, cor por classe e a diagonal $y = x$.

A hipótese será confirmada se o Wilcoxon unilateral sobre as 408 instâncias favorecer o MSPD-LS com $p < 0{,}05$ e seu *gap* médio for menor ou igual ao do MSPD em todas as classes; parcialmente confirmada se $p < 0{,}05$, mas o MSPD-LS for pior em alguma classe; e refutada se $p \geq 0{,}05$.

### 3.3.2. Hipótese 2: Metaheurísticas propostas versus o controle MSPD

A hipótese H2 afirma que, com o mesmo tempo, VNS, ILS, ALNS e MA obtêm *gap* ao BKS significativamente menor que o MSPD, e que o MA, por combinar recombinação e busca local, é o melhor de todos.

Para testá-la, MSPD, MSPD-LS, VNS, ILS, ALNS e MA serão executados nas 408 instâncias, coletando-se o *gap* médio, máximo e da melhor semente, o desvio-padrão entre sementes, a taxa de execuções que atingem o BKS, o tempo até a melhor solução e o *rank* médio (1 = menor *gap* na instância).

Os resultados serão apresentados em uma tabela com uma linha por método e uma coluna por medida; em uma tabela com o Wilcoxon de cada metaheurística contra o MSPD e do MA contra os demais métodos propostos (vitórias/empates/derrotas e valor-$p$ bruto e corrigido); em um *boxplot* do *gap* por método; e em uma curva de distribuição acumulada, com a tolerância $\tau \in [0, 5]\%$ no eixo $x$, a porcentagem de instâncias com *gap* $\leq \tau$ no eixo $y$ e uma curva por método.

A hipótese será confirmada se o Friedman tiver $p < 0{,}05$ e as quatro metaheurísticas vencerem o MSPD com $p_{Holm} < 0{,}05$ (família de quatro comparações); parcialmente confirmada se apenas parte delas vencer; e refutada se nenhuma vencer. A parte "o MA é o melhor" será confirmada se ele tiver o menor *rank* médio e vencer MSPD-LS, VNS, ILS e ALNS com $p_{Holm} < 0{,}05$ (segunda família de quatro comparações).

### 3.3.3. Hipótese 3: Distância ao ótimo

A hipótese H3 afirma que, nas instâncias com ótimo provado, a melhor metaheurística proposta (a de menor *rank* médio em H2) tem *gap* médio ao ótimo de no máximo 0,5% e atinge o ótimo em pelo menos 90% das execuções.

Para testá-la, a formulação matemática da Seção 1.2 será resolvida no HiGHS nas classes T1 a T3, com limite de 3.600 s, por geração iterativa de restrições: resolve-se o modelo sem as restrições de subciclo e, enquanto a solução tiver mais de um ciclo, acrescentam-se cortes que os eliminam, o que fornece um limitante superior válido mesmo sem provar a otimalidade. Serão coletados a situação final e o tempo de resolução, o ótimo ou limitante superior, o *gap* ao ótimo das execuções de H2 e a taxa de execuções

ótimas; como a resolução exata só deve terminar nas instâncias menores, a conclusão não se estende às classes T4 e PK.

Os resultados serão apresentados em uma tabela com uma linha por método e colunas com o *gap* médio e máximo ao ótimo e a porcentagem de execuções ótimas, por classe (T1 a T3) e no total, e em uma tabela com uma linha por classe e colunas com o número de instâncias, o número resolvido à otimalidade, o tempo médio de resolução e a distância média entre BKS e limitante nas não resolvidas. A hipótese será confirmada se os dois limiares forem atingidos; parcialmente confirmada se apenas o de *gap* for atingido; e refutada se o *gap* médio ao ótimo passar de 0,5%. Se a otimalidade for provada em menos de 30 instâncias, a avaliação será apenas descritiva.

### 3.3.4. Hipótese 4: Clusters espalhados exigem a troca do representante do cluster

A hipótese H4 afirma que, com *clusters* espalhados e distâncias não métricas (PK), o ILS, cuja perturbação só altera os representantes dos *clusters* situados no trecho removido, perde mais desempenho que o VNS e o ALNS, capazes de trocar o representante de qualquer *cluster*, e que o ILS-C corrige essa perda.

Para testá-la, serão usados os resultados de H2 nas classes T4 (TSPLIB de tamanho semelhante) e PK, acrescidos do ILS-C com as mesmas 10 sementes nessas classes, coletando-se o *gap* médio de cada método por classe e a degradação (*gap* em PK menos *gap* em T4).

Os resultados serão apresentados em uma tabela com uma linha por método (ILS, ILS-C, VNS, ALNS e, como referência, MA) e colunas com o *gap* em T4, o *gap* em PK e a degradação, e em um gráfico de barras agrupadas com os métodos no eixo $x$, o *gap* médio no eixo $y$ e uma barra por classe.

A hipótese será confirmada se (i) a degradação do ILS for maior que a do VNS e a do ALNS (comparação descritiva) e (ii) o ILS-C tiver *gap* menor que o ILS na classe PK (Wilcoxon unilateral, $p < 0{,}05$); parcialmente confirmada se só (i) for atendida; e refutada se (i) falhar. Como PK tem apenas 18 instâncias e difere de T4 em mais de um aspecto, um eventual efeito será atribuído à combinação dessas características, e não a uma delas isoladamente.

## 4. Cronograma

A execução da metodologia da Seção 3 foi dividida nas nove etapas numeradas da Tabela 1, que vão da infraestrutura comum à redação do artigo final. As etapas E1 e E2 preparam o código comum (leitura de instâncias, verificador de soluções e BL) e as instâncias; E3 a E5 implementam o controle, os métodos propostos e a referência exata; E6 corrige e verifica as implementações em testes piloto, sem ajuste de parâmetros; E7 executa os experimentos; E8 analisa os resultados de H1 a H4; e E9 redige o artigo final.

A equipe tem cinco integrantes, e a organização da implementação aloca ao menos uma das heurísticas das Seções 3.1 e 3.2 para implementação individual. Gabriel Coelho dos Santos implementa o ALNS e apoia a preparação das instâncias TSPLIB-GCOP (E2); Inaue Ferreira da Silva implementa o ILS e o ILS-C e apoia a análise estatística (E8); Kayque Meira Siqueira implementa a BL e o MA e apoia a infraestrutura (E1); Lucas

**Tabela 1. Etapas, parcela do tempo total, duração e período (em dias corridos).**

| Etapa | Descrição | Parcela | Dias | Período |
|---|---|---|---|---|
| E1. Infraestrutura | Leitura de instâncias, verificador e BL | 8% | 6 | 1–6 |
| E2. Instâncias | TSPLIB-GCOP, cálculo de $B$ e geração POKE | 8% | 6 | 1–6 |
| E3. Controle | $\sigma$, MSPD e Formulação Exata | 11% | 8 | 7–14 |
| E4. Propostas | VNS, ILS, ILS-C, ALNS, MA e MSPD-LS | 16% | 12 | 7–18 |
| E5. Formulação exata | Modelo matemático da Seção 1.2 no HiGHS | 11% | 8 | 7–14 |
| E6. Testes piloto | Correção e verificação | 5% | 4 | 19–22 |
| E7. Experimentos | Formulação exata e heurísticas | 19% | 14 | 15–28 |
| E8. Análise | Testes, tabelas e gráficos de H1 a H4 | 8% | 6 | 29–34 |
| E9. Escrita | Redação do artigo final | 14% | 10 | 31–40 |

Fernandes Santos implementa o VNS e apoia a geração das instâncias POKE (E2); e Rubens da Cunha Castro implementa o MSPD e o MSPD-LS e apoia a formulação exata (E5) e a execução dos experimentos (E7).

## Referências


[1] Eduardo Álvarez-Miranda, Martin Luipersbeck, and Markus Sinnl. Gotta (efficiently) catch them all: Pokémon GO meets orienteering problems. *European Journal of Operational Research*, 265(2):779–794, 2018.

[2] Claudia Archetti, Francesco Carrabs, and Raffaele Cerulli. The set orienteering problem. *European Journal of Operational Research*, 267(1):264–272, 2018.

[3] G. A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812, 1958.

[4] Matteo Fischetti, Juan José Salazar-González, and Paolo Toth. A branch-and-cut algorithm for the symmetric generalized traveling salesman problem. *Operations Research*, 45(3):378–394, 1997.

[5] Helena R. Lourenço, Olivier C. Martin, and Thomas Stützle. Iterated local search. In Fred Glover and Gary A. Kochenberger, editors, *Handbook of Metaheuristics*, pages 320–353. Kluwer, 2003.

[6] N. Mladenović and P. Hansen. Variable neighborhood search. *Computers & Operations Research*, 24(11):1097–1100, 1997.

[7] Pablo Moscato. On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms. Technical Report C3P 826, Caltech, 1989.

[8] Ilhan Or. *Traveling Salesman-Type Combinatorial Problems and Their Relation to the Logistics of Regional Blood Banking*. PhD thesis, Northwestern University, 1976.

[9] Robert Pěnička, Jan Faigl, and Martin Saska. Variable neighborhood search for the set orienteering problem and its application to other orienteering problem variants. *European Journal of Operational Research*, 276(3):816–825, 2019.

[10] Stefan Røpke and David Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455–472, 2006.

[11] Zülal Sevkli and F. Erdoğan Sevilgen. Variable neighborhood search for the orienteering problem. In *Computer and Information Sciences – ISCIS 2006*, volume 4263 of *LNCS*, pages 134–143. Springer, 2006.

[12] Hiromitsu Takahashi and Akira Matsuyama. An approximate solution for the Steiner problem in graphs. *Mathematica Japonica*, 24(6):573–577, 1980.

[13] T. Tsiligirides. Heuristic methods applied to orienteering. *Journal of the Operational Research Society*, 35(9):797–809, 1984.

[14] Pieter Vansteenwegen, Wouter Souffriau, Greet Vanden Berghe, and Dirk Van Oudheusden. Iterated local search for the team orienteering problem with time windows. *Computers & Operations Research*, 36(12):3281–3290, 2009.